

ECE 5760 Lab 1: Hardware ODE solver with HPS control

Md Shad, Michael Grillo, Stephen Barnett

February 13, 2026

1 Introduction

In this first lab, we implemented a hardware-based ordinary differential equation (ODE) solver on the DE1-SoC FPGA to integrate the Lorenz system utilizing a Digital Differential Analyzer (DDA). The solver we implemented was a first-order Euler integration method which updates the system's three coupled state variables, which are $x(t)$, $y(t)$, and $z(t)$. This is done in fixed-point arithmetic, allowing us to do a real-time computation of the chaotic Lorenz attractor. We created a design where you can configure parameter values and initial conditions so an user can explore and test the system's sensitivity to small perturbations and highlighting the concept of deterministic yet unpredictable behavior.

The DDA core was implemented in synthesizable Verilog and executed entirely in FPGA hardware, while the Hard Processor System (HPS) provided high-level control and visualization. The HPS communicated with the solver through memory-mapped parallel I/O (PIO) interfaces, issuing commands to start, pause, reset, and adjust integration parameters. Solver outputs were streamed to the HPS and rendered on a VGA display, which plotted all three two-dimensional projections of the Lorenz attractor in real time.

We also had a command-line interface running on the HPS, which allows users to dynamically modify the Lorenz parameters (σ , β , and ρ), integration timestep, and initial conditions during execution. The system also supported real-time control of animation speed with speed up and slow down features, pausing and resuming integration, and clearing the display without breaking the computation. Through careful fixed-point design, hardware–software co-design, and verification using ModelSim and SignalTap, the final implementation demonstrated accurate numerical integration, responsive visuals, and efficient and accurate coordination between FPGA logic and the HPS.

2 Mathematical Background

2.1 Lorenz System

The Lorenz system consists of three nonlinear ordinary differential equations.

$$\frac{dx}{dt} = \sigma(y - x) \quad (1)$$

$$\frac{dy}{dt} = x(\rho - z) - y \quad (2)$$

$$\frac{dz}{dt} = xy - \beta z \quad (3)$$

The Lorenz system defined above follows the formulation presented in the ECE 5760 laboratory handout [1].

The variables $x(t)$, $y(t)$, and $z(t)$ help describe and define the state of the Lorenz system at time t . The constants σ , ρ , and β control the behavior of the system.

Each parameter has a physical and mathematical meaning within the Lorenz model:

σ The parameter σ controls how strongly x is driven toward y . It determines the rate at which the horizontal velocity difference equalizes. Larger values cause x to respond more quickly to changes in y .

ρ The parameter ρ represents the temperature difference driving convection and acts as the main energy input to the system. When ρ is small the motion settles to a steady state, while larger values can produce oscillatory and eventually chaotic behavior.

β The parameter β is a damping term that limits growth in the z direction. It models vertical heat loss and prevents the system from diverging to infinity.

The initial conditions in our design, $x(0)$, $y(0)$, and $z(0)$ are the starting point of the trajectory in state space.

Because the Lorenz system is chaotic, very small changes in these starting values produce completely different long-term trajectories, even though the governing equations remain the same.

The timestep Δt determines how far forward in time the Euler integrator advances on each iteration. A smaller timestep improves numerical accuracy but requires more iterations, while a larger timestep runs faster but can introduce numerical error or instability.

2.2 Discrete-Time Update Equations

The FPGA operates in discrete clock cycles, so we had to make sure the continuous-time equations must be approximated using numerical integration. We used the forward Euler method, which updates the system by making a small step in time at each iteration.

At time step k , the current state of the system is (x_k, y_k, z_k) . The next state is computed using

$$x_{k+1} = x_k + \Delta t \cdot \sigma(y_k - x_k) \quad (4)$$

$$y_{k+1} = y_k + \Delta t \cdot (x_k(\rho - z_k) - y_k) \quad (5)$$

$$z_{k+1} = z_k + \Delta t \cdot (x_k y_k - \beta z_k) \quad (6)$$

In hardware, the FPGA uses the current values of x , y , and z , multiplies them by the timestep Δt , and adds the result onto the previous state. Each clocked update therefore advances the simulation forward by a small amount of time.

2.3 Properties of the Lorenz System

For these parameter values the Lorenz system displays chaotic behavior. The equations are deterministic, meaning the next state is completely determined by the current state, yet long-term prediction becomes impossible because very small differences in the starting conditions grow rapidly over time.

Instead of settling to a constant value or repeating pattern, the trajectory remains bounded while continually moving through space. The resulting shape is known as a **strange attractor** and appears as a butterfly-like structure when plotted.

This property makes the Lorenz system a useful test of a numerical solver. Small numerical errors from fixed-point rounding or timing differences quickly change the trajectory, so correct operation of the hardware solver can be verified visually from the plotted curve.

Because the Euler method is a first-order numerical approximation, smaller timesteps improve stability and prevent divergence of the solution in fixed-point arithmetic.

3 Roles of the ARM (HPS) and FPGA

The design used a hardware–software co-design approach in which the FPGA executed the real-time computation while the ARM-based Hard Processor System (HPS) managed control and communication. The FPGA was assigned to deterministic, high-throughput numerical computation, while the ARM processor provided system-level control, user interaction, data management, and graphics rendering. This separation allowed the Lorenz system to be integrated in real time without burdening the processor with strict timing constraints.

FPGA Responsibilities The FPGA implemented the core numerical solver using a Digital Differential Analyzer. At each integration step, the FPGA computed the Lorenz system derivatives and updated the state variables using a first-order Euler method. All arithmetic was performed using signed fixed-point representations to efficiently map onto the Cyclone V DSP blocks and to ensure predictable latency. The use of hardware multipliers enabled parallel evaluation of terms such as xy , $(\rho - z)x$, and $\sigma(y - x)$, which would be comparatively expensive in software.

By executing the integration loop entirely in hardware, the FPGA provided deterministic timing behavior and consistent numerical update rates independent of processor load. This was especially important for chaotic systems such as the Lorenz attractor, where small timing variations or dropped updates could significantly alter the resulting trajectory. The solver advanced only when explicitly clocked or enabled, allowing precise synchronization with the visualization and user commands.

The FPGA also implemented memory-mapped parallel I/O (PIO) registers that served as the interface to the HPS. These registers exposed the current state variables (x , y , and z) and accepted configuration values including the Lorenz parameters (σ , β , ρ), the integration

timestep, and control signals such as reset and run/stop. Fixed-point results were sign-extended in hardware to 32-bit values to simplify software-side access and maintain numerical correctness.

ARM (HPS) Responsibilities The ARM processor running on the HPS acted as the system orchestrator. Its primary role was to configure and supervise the hardware solver rather than perform numerical integration itself. Through memory-mapped I/O writes, the ARM initialized the solver state, updated parameters, and controlled the execution flow of the DDA. This allowed the user to modify parameters dynamically, reset the system, or pause and resume integration without reprogramming the FPGA.

In addition to control, the ARM was responsible for data consumption and visualization. It periodically read the solver outputs from the FPGA and mapped the numerical results to screen coordinates for display. The ARM handled all VGA drawing operations, including plotting the three orthogonal projections of the Lorenz attractor, clearing the screen, and overlaying textual information such as parameter values and system status. Offloading graphics to the processor avoided unnecessary complexity in the FPGA design while leveraging the flexibility of software for rendering and user interface tasks.

The ARM also implemented a command-line interface that allowed interactive control of the system at runtime. User commands were parsed in software and translated into appropriate hardware register updates, enabling real-time experimentation with different parameter sets and initial conditions. This interface provided a convenient and extensible mechanism for interacting with the solver without introducing additional hardware inputs.

System-Level Integration The separation between computation and control resulted in a clean and scalable architecture. The FPGA operated as a deterministic numerical engine, while the ARM provided adaptability, debugging capability, and user-facing features. Communication between the two domains occurred exclusively through well-defined memory-mapped registers, which simplified verification and debugging. Hardware correctness was verified independently in ModelSim and SignalTap, while software behavior could be modified and tested without altering the hardware design.

Overall, this division of labor allowed the system to combine the performance and predictability of hardware with the flexibility and ease of development offered by software. The resulting design demonstrates an effective approach to real-time numerical simulation on an FPGA-based system-on-chip platform.

4 RTL Design

This design implements one update of the Lorenz system across three separate dimensions (x , y , z) each clock cycle, computed through the use of 7.20 fixed-point arithmetic. We chose to use this resolution of fixed point to balance the range of possible values with maximum precision. When we cleverly scale values mid computation, we end up with integrated values not exceeding $|64|$, allowing us to store signed numbers within this range, leaving the other 20 bits for as high resolution as possible in fixed point, while leaving each 27 bits, the largest

multiplicate value within the DSP blocks on the board. We chose to use fixed-point as fixed-point allowed for a very simple way to store values and to update values that just required an initial scale by 2^{20} rather than the significantly more complex hardware needed to support floating-point arithmetic, which for a Lorenz system would product the same macroscopic output. This RTL design followed a similar format to the hardware block diagram sourced from the Lab Handout. This included an organization into three state registers, which were implemented via three instances of the integrator module, and a series of prior calculations to derive values for dx, dy, and dz, with the effective timestep implemented through distributed right shifts (in this case bit-shifted by 8 to reduce logic).

4.1 Signed Multiply

The first commonly module that was used for this Lorenz Integrator was a signed multiplier for fixed bit calculations provided in the lab handout's "code snippet". This multiplier took two 27 bit inputs and returned a 27 bit output, the maximum multiply operation that can occur with one DSP block on the Altera DE1-SoC. As truncation must occur in this multiplication, an intermediate 54 bit wire was assigned the output of $a * b$, which was then truncated to maintain an equal number of fractional bits and to keep the sign bit to ensure proper signed arithmetic. In the case of our 7.20 two's complement representation,

```
assign out = {mult_out[53], mult_out[45:20]};
```

where `mult_out[53]` is the sign bit of the result and `mult_out[45:20]` represents the numerical portion of the result considering the fixed-point representation. This special multiplication was used to ensure that our lorenz integrations values would retain proper values after arithmetic. One issue; however, that arose with this multiply was a high sensitivity to overflow that occurred when calculating update values. As covered in the following section.

4.2 Calculation of Update Values

As specified mathematically, the prescaled dx, dy, and dz values were calculated through a series of addition, subtraction, and multiplications, defined by the following equations: $dx = \sigma(y - x)$, $dy = x(\rho - z) - y$, $dz = xy - \beta z$. To preform these equations, a series of adders, subtractors and signed multipliers were to be used, followed by a right shift of 8, denoted by " $>>> 8$ ", which would effectively multiply each value by $1/256$, the scaling value of dt. In practice however, large multiplications, such as a βz or $x(\rho - z)$ multiplication would often result in overflow in the signed multiplier due to the limited dynamic range of the 7.20 fixed point representation. This led to overflow values which where significantly different from expected values and degraded the calculation of the Lorenz system. To fix such issue, a series of synthetic bit shifts for dt scaling were added intermediately in the calculation process of dxdt, dydt, and dzdt, which prevented overflow. These bit-shifted calculations were as follows: $dxdt = \sigma((y - x) >>> 8)$, $dydt = (((x >>> 8)(\rho - z)) - (y >>> 8))$, $dzdt = ((x >>> 8)y - \beta(z >>> 8))$. These new dxdt, dydt, and dzdt values were now ready to be passed into the accumulation step in the integrator by digital direct analyzer, which will be covered in the next subsection.

4.3 Integration by Digital Direct Analyzer

Each state variable in the system is implemented using a Digital Differential Analyzer (DDA) integrator, which was instantiated as synchronous RTL modules. The DDA integrator used to calculate the Lorenz system performs a first-order Euler integration by accumulating a time-scaled derivative value into a state register sequentially. A block diagram of such system sourced from the Lab Handout is shown below. Such system includes as previously mentioned a state register to store the current value, which gets accumulated with an calculated value, which is multiplied by a corresponding timestep, to which the value is added to the previous register value. At this point, to allow for signal reset, a mux is inserted to allow for the possibility to return to initial conditions in the case of a reset signal. This integrator is instantiated three separate times, once for x , once for y , and once for z values. The values from this system are transmitted to the ARM HPS and additionally routed back for further accumulation. The integrator takes in a initial value, a clock signal, reset signal, and an accumulation function (for example $dxdt$), and operates synchronously.

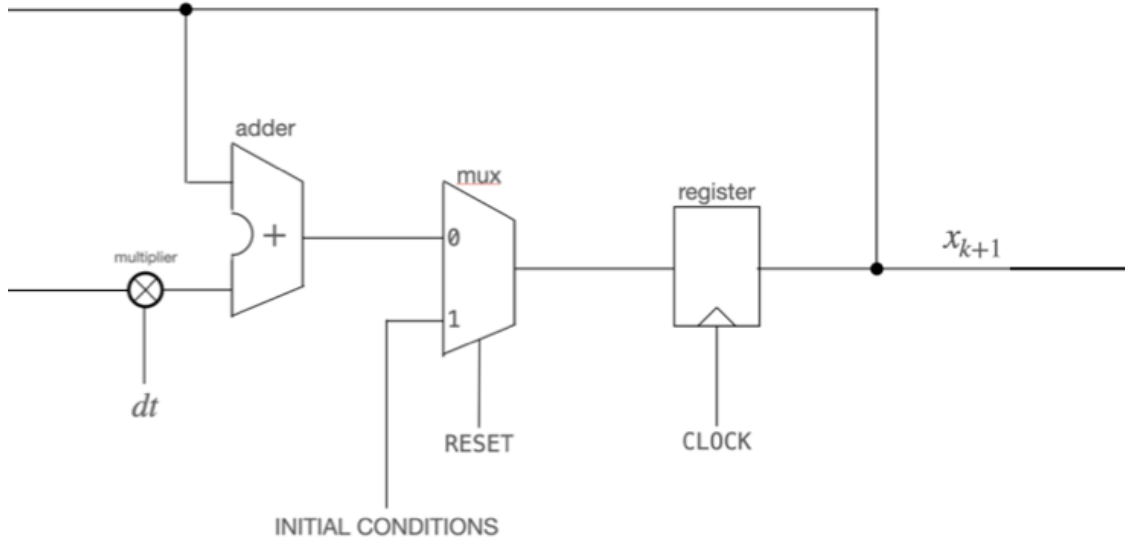


Figure 1: Block diagram of the DDA integrator used to implement Euler integration of the Lorenz system (adapted from [1]).

The RTL for the Lorenz system calculation includes inputs for σ, β, ρ, dt alongside $xk_initial, yk_initial, zk_initial, clk$, and $reset$, and returns a continuous update of xk, yk , and zk . Three parallel operations are occurring in each clock cycle, which includes recalculating $dxdt, dydt$, and $dzdt$, then accumulating the previous values with these new values than, unless $reset$, updating the registers with new values for xk, yk , and zk . Three total integrators are used in this Lorenz integrator, alongside four signed multiply modules, greatly within the scope of DSP and LUT availability on the DE1-SoC.

5 Design and Testing Methods

5.1 Hardware Testing

The primary form of testing our Verilog design was done by comparing the results of our simulation with the output of a Python Lorenz implementation. We then verified that the ModelSim results matched what was actually happening on the FPGA by using SignalTap (see figure 2). A basic testbench was instantiated in Verilog, beginning by setting initial inputs as the following: $dxdt = \sigma((y - x) >>> 8)$, $dydt = (((x >>> 8)(\rho - z)) - (y >>> 8))$, $dzdt = ((x >>> 8)y - \beta(z >>> 8))$. By shifting midway through calculating dx, dy, dz and integrating the dt through this process, we were able to eliminate this overflow leading to the ModelSim and SignalTap plots shown above.

```
dt = (1./256)
x = [-1.]
y = [0.1]
z = [25.]
sigma = 10.0
beta = 8/3
rho = 28.0
```

inside of an instantiated Device Under Test in our non-synthesizable Verilog, where a manual clock was initialized and our system was automatically reset to provide a clean starting point of our initial conditions. Through simulated clock cycles, the Lorenz integrator was able to be verified through the output from ModelSim as shown in 3.

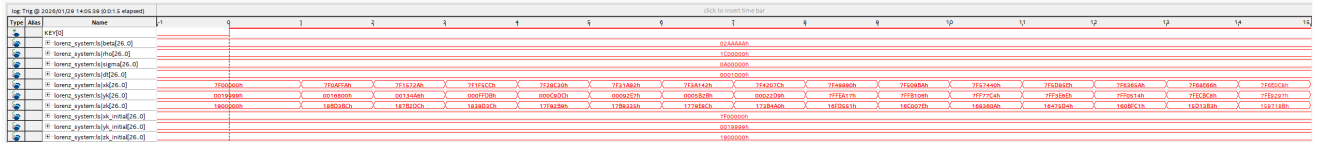


Figure 2: SignalTap Output for Lorenz Integrator

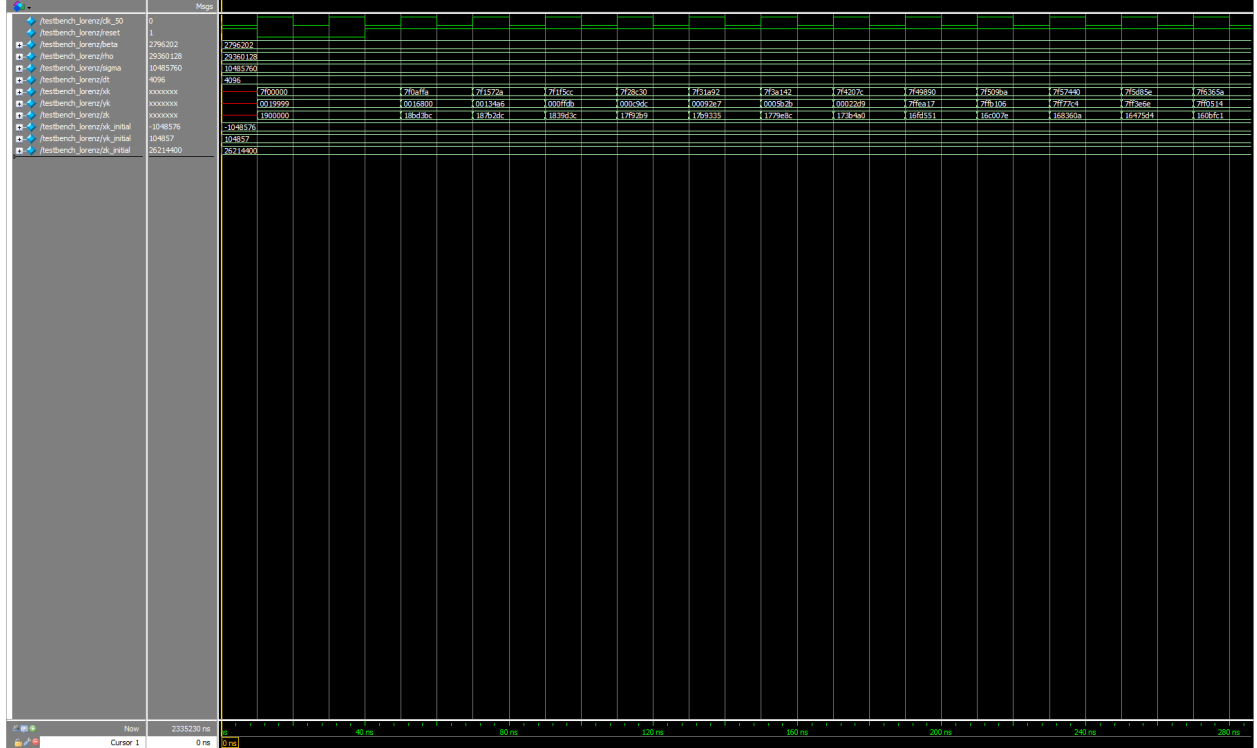


Figure 3: ModelSim Output for Lorenz Integrator

We noticed that the output of the Python integrator matched our implementation for the first few cycles, until it diverged quickly. As discussed in lecture, this is because the Python implementation used floating point math to compute the curve, and our hardware uses fixed-point. This difference in precision results in the initial conditions to be slightly different, something that the Lorenz system is particularly sensitive to. Because the results appeared 'close enough,' combined with the fact that our plots appear correct, as shown in 4, we were confident that our hardware is correct.

This 'close enough' verdict provided by our results is an expected feature of the Lorenz system, where as a chaotic system, small changes in inputs lead to a infinitely different changes in the output. In this such case it is expected that small inconsistencies in fixed vs floating point representations will quickly diverge from one another and through confirmation of an initial set of data points such as the matching of the first set of data points across fixed and floating point, allows us to ensure the correctness of our Lorenz system, as this difference in representation forced a shift from quantitative to qualitative measures as accumulation continued. This testing strategy agreed with the known Lorenz dynamics to ensure proper functionality of our Lorenz integrator

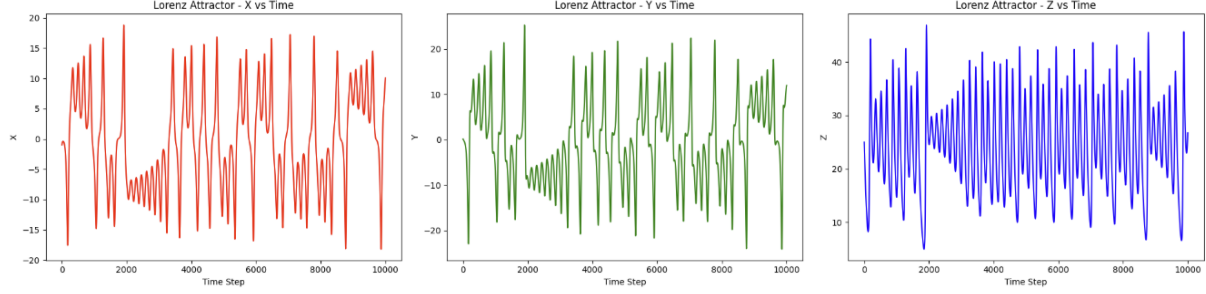


Figure 4: x vs t, y vs t, z vs t Python Plots

As described in the previous section, while testing our implementation initially through ModelSim, we encountered a phenomena where our intregration patterns didn't follow the chaotic action as shown in the python plots but rather followed a large "charging" pattern where after strong initial actions, the values remained fairly constant. We determined that this was a result of overflow in our pre-integration steps of calculating dx , dy , and dz . This led to the change listed above where we

In addition to full-system testing, individual RTL components were verified through closer inspection of our system. It was ensured that not only, plots looked visually correct, but that each module was ensuring correct calculations in hardware. Reset behavior was tested by asserting a reset deterministically in our testbench to ensure that initial values were presented in the event of a reset. After preforming such reset, it was verified that proper integration occurred.

5.2 Software Testing

The software for this assignment was only responsible for taking the outputs from the hardware and drawing it to the screen. To ensure correctness, we had to verify that we were (1) correctly communicating with the FPGA for each parameter and (2) drawing the plot to the screen accurately.

We started by hard coding a C program that grabs the value on each output (x , y , and z) and serving it to the serial output with a simple *printf* call in a loop. We were able to compare this output with the output we saw in both ModelSim and SignalTap to verify that we were seeing the actual output. Once we implemented the interactive loop that allows users to manipulate the parameters, we verified that the HPS was able to modify the outputs (σ , β , ρ , initial conditions) by changing them, resetting the program, and observing the differences in output.

Graphically, we verified our software by allowing it to run continuously for several minutes under different initial conditions, and stress testing the interactive components. Some of these stress tests included maximum speed, minimum speed, high-frequency resets, changing parameters without a reset, and more.

6 Results

7 Chip Planner Layout

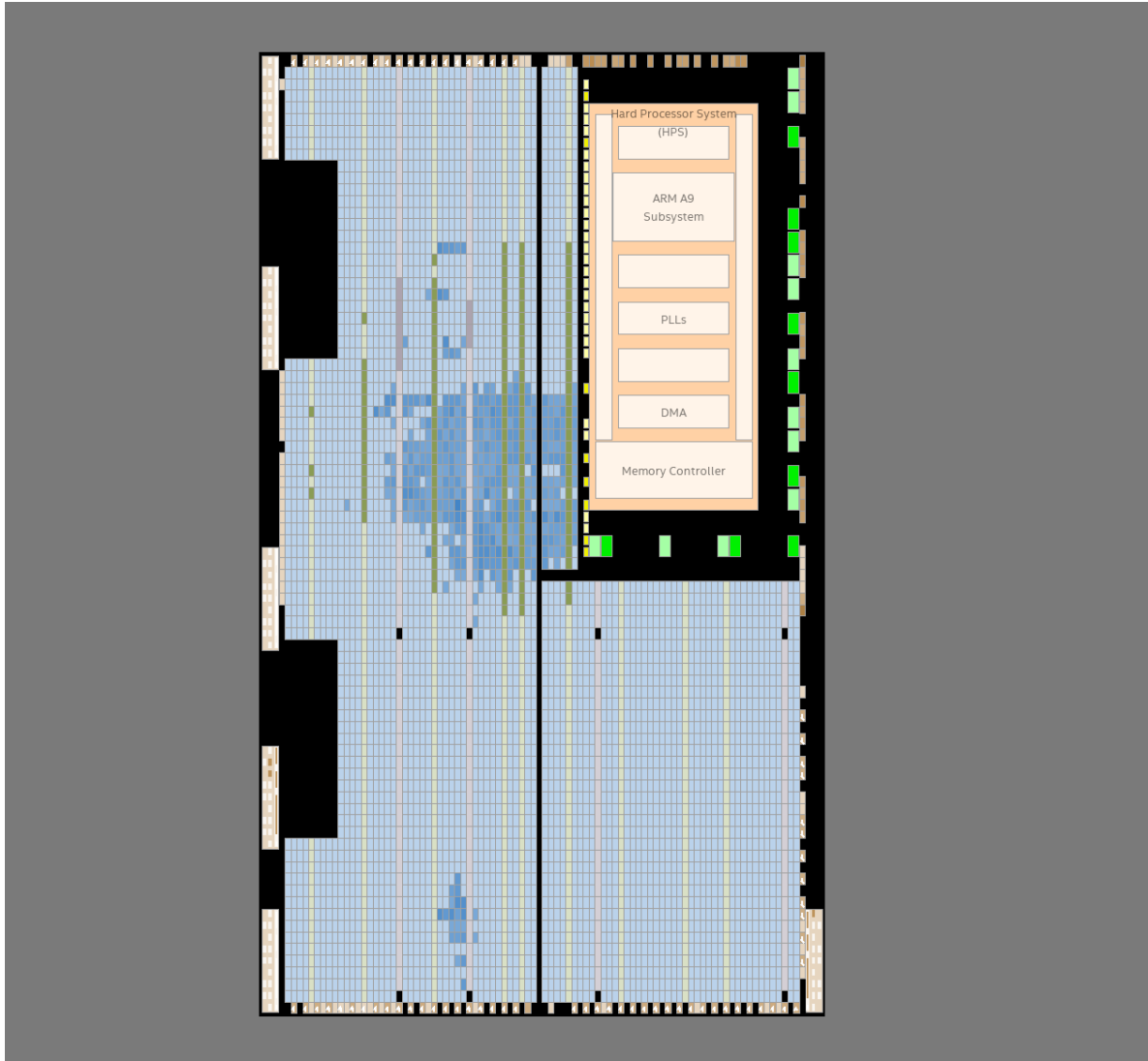


Figure 5: FPGA Chip Planner view showing placement of the system modules and routing resources.

The Chip Planner view provides a physical representation of how the synthesized design is mapped onto the FPGA fabric. Unlike the RTL schematic, which shows logical connectivity, the chip planner shows the *actual hardware implementation* after synthesis, placement, and routing.

Each block in the figure corresponds to a placed module (such as compute units, memory blocks, and interconnect logic), and their locations reflect how the CAD tool distributed logic across the available adaptive logic modules (ALMs), routing channels, and embedded

resources such as block RAMs and DSP units. The routing lines illustrate how signals physically travel between modules on the chip.

This view is significant because FPGA performance is not determined only by the correctness of the Verilog design but also by physical placement and routing. Poor placement can lead to long routing paths, increased propagation delay, and timing violations. By examining the Chip Planner, we can verify that:

- modules are distributed rather than congested in one region,
- memory and compute units are placed near each other to reduce routing delay,
- critical interconnect paths do not span large portions of the chip.

Therefore, the Chip Planner confirms that the design successfully maps to real hardware resources and helps diagnose timing issues, routing congestion, and performance bottlenecks that are not visible at the RTL level.

8 Hardware Resource Utilization

Quartus Prime reports that our synthesized design uses 23,076 ALMs out of 32,070 available (72%), 29,639 registers, 2,382,234 block memory bits out of 4,065,280 (59%), and 15 DSP blocks out of 87 (17%). In addition, the design uses 368 I/O pins out of 457 (81%), 3 PLLs out of 6 (50%), and 1 DLL out of 4 (25%).

These hardware usage results directly correspond to the architecture of our Lorenz attractor visualization system, which consists of a hardware Euler integrator implemented in Verilog and a VGA graphics subsystem controlled by the HPS.

The high ALM (logic element) utilization is primarily caused by the arithmetic datapath in the `lorenz_system` module and the VGA video generation logic. The Lorenz equations require multiple additions, subtractions, and comparisons every clock cycle (e.g., $y - x$, $\rho - z$, and accumulation inside the integrator). In addition, the VGA drawing routines continuously compute pixel coordinates and boundaries for rendering points and trajectories. These operations are implemented using combinational logic and therefore consume a large portion of the FPGA’s programmable logic fabric.

The large register count is expected because the system is fully synchronous. Each state variable (x , y , and z) is stored in registers inside the `integrator` module:

Every clock cycle, the new value is written back into the state register. Additional registers are also required for video timing counters, pixel coordinates, and pipeline stages in the arithmetic units. As a result, nearly thirty thousand flip-flops are used.

The DSP block usage comes from the nonlinear terms of the Lorenz equations. In particular, the products xy , xz , and βz are computed using the `signed_mult` module:

```
assign mult_out = a * b;
assign out = {mult_out[53], mult_out[45:20]};
```

Quartus maps these multiplications onto dedicated hardware multipliers (DSP blocks), which are much faster and more area-efficient than implementing multiplication using logic

gates. Since the Lorenz system requires only a few multiplications per iteration, only 15 of the available 87 DSP blocks are used, leaving significant computational headroom.

The block memory usage is mainly associated with the VGA graphics system and HPS communication. The framebuffer and video timing logic require on-chip memory to store pixel information and maintain continuous display output. This explains the moderate BRAM utilization of 59%.

Finally, the PLLs are used to generate multiple clock domains. The system requires separate clocks for the FPGA system logic and the VGA pixel clock, so multiple phase-locked loops are necessary to produce the correct frequencies.

8.1 Critical Path

In our design, the longest combinational logic chain is expected to live inside the Lorenz derivative datapath (the `lorenz_system` module), where multiple fixed-point operations are performed in a single cycle to compute `dxdt`, `dydt`, and `dzdt`. In particular, the `dydt` and `dzdt` expressions contain the deepest arithmetic chains because they involve both a multiplication and additional add/subtract stages:

$$\text{dydt} = ((x \ggg 8)(\rho - z)) - (y \ggg 8), \quad \text{dzdt} = (x \ggg 8)y - \beta(z \ggg 8).$$

These paths include: (1) subtraction to form $(\rho - z)$, (2) at least one 27×27 signed multiply mapped to a DSP block, and (3) one or more add/subtract operations to combine terms. Even when the multiplication itself is implemented in a dedicated DSP block, the surrounding sign-extension, truncation, shifting, and adder logic still contributes to the overall delay, especially when chained.

As a result, we expect the critical path to be a register-to-register path from the current state registers (x_k, y_k, z_k) through the combinational derivative logic (subtract $\rightarrow$ multiply $\rightarrow$ add/subtract $\rightarrow$ shift/truncate) and into the next-state accumulation input of the `integrator` registers.

9 Documentation

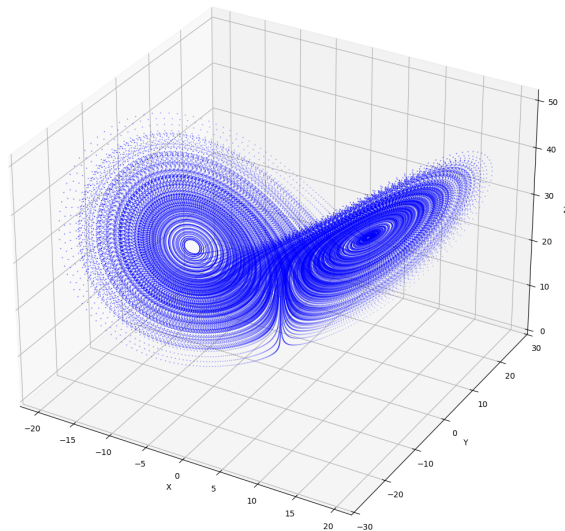


Figure 6: 3D Plot generated from our SignalTap output.

We captured the values for the x , y , and z state variables from the FPGA using SignalTap to plotted to reconstruct the Lorenz attractor. The characteristic butterfly shape confirms that the hardware integrator is producing the correct system dynamics.



Figure 7: VGA Display produced by the HPS on the DE1-SOC.

This was our final demo displayed on the FPGA, demonstrating all the parameters a user

can modify on the serial monitor and the resulting graphs based on those set parameters by the users.

9.1 Program Listings

Our C program uses several macros that may appear cryptic without explanation. To reduce this confusion we have included a detailed listing of the most important macros used in our program. This includes some macros that were not created by us, but instead by Hunter and Bruce in the VGA Color Primitive demo code. We have included these macros in our listing for completion. See Table 1 and Table 2 for these listings.

Table 1: Memory Map and Hardware Address Regions

Region / Symbol	Description
SDRAM_BASE	Physical base address of SDRAM pixel buffer (0xC0000000).
SDRAM_END	End physical address of SDRAM region (0xC3FFFFFF).
SDRAM_SPAN	Total size of SDRAM region (0x04000000).
FPGA_CHAR_BASE	Base address of FPGA character buffer (0xC9000000).
FPGA_CHAR_END	End address of character buffer (0xC9001FFF).
FPGA_CHAR_SPAN	Character buffer size (0x00002000).
HW_REGS_BASE	Base address for HPS lightweight (LW) hardware registers (0xFF200000).
HW_REGS_SPAN	Address span for LW register mapping (0x00005000).
*_OFFSET	Offsets relative to HW_REGS_BASE: • 0x10 — X register • 0x20 — Y register • 0x30 — Z register • 0x40 — σ parameter • 0x50 — β parameter • 0x60 — ρ parameter • 0x70, 0x80, 0x90 — Initial X/Y/Z values • 0x100 — Control register (clock/reset bits)

Table 2: Software Macros and Helper Utilities

Macro / Symbol	Description
SCALE	Fixed-point scale factor 1048576 (2^{20}) used for float fixed conversion.
VGA_PIXEL(x,y,color)	Writes a 16-bit RGB565 pixel to the VGA buffer at (x, y) using $((y)*640 + (x)) \ll 1$ address calculation.
RESET_HIGH	Assert reset (set bit 1 of control register).
RESET_LOW	De-assert reset (clear bit 1).
CLK_HIGH	Set clock bit (bit 0).
CLK_LOW	Clear clock bit (bit 0).
float2fix(a)	$(\text{int})(a \times 2^{20})$
fix2float(a)	$(\text{float})(a)/2^{20}$
int2fix(a)	$a \ll 20$
fix2int(a)	$a \gg 20$

9.2 User Interface

We implemented a simple command line interface (CLI) for changing parameters and adjusting the speed of our solvers. We run our CLI in a separate *pthread* which reads user input from a blocking *scanf* call, and handles it in a case statement. This thread loops indefinitely until the main thread exits (ctrl-C).

The CLI prompts the user to first enter one of several options:

r/b/s/x/y/z/start/stop/reset/[(speed up)](slow down)

The single character parameters stand for Rho, Beta, Sigma, Initial X, Initial Y, Initial Z. The user will enter the corresponding character, then they will be prompted to enter the new value. The change will be reflected immediately in the parameter list on the VGA display. This interface also enables pausing/unpausing, resetting, and adjusting speed. These parameters do not require a second input and will take effect immediately upon hitting enter.

Table 3: Program Listing Table: Mandelbrot Software Control Program

Module / Block	Type	Description
<code>control_thread</code>	Software Thread	Keyboard control thread for interactive Mandelbrot navigation. Uses <code>ncurses</code> to read user key presses and updates the current complex-plane viewing window by changing <code>cr_range</code> , <code>ci_range</code> , <code>cr_start</code> , and <code>ci_start</code> . Supports zooming in/out and panning across the fractal.
<code>main</code>	Software Control / Host Interface	Main host-side control routine. Opens <code>/dev/mem</code> , maps the lightweight HPS-to-FPGA bridge into virtual memory, initializes pointers to memory-mapped FPGA control registers, launches the control thread, and continuously writes updated Mandelbrot view parameters to the FPGA. Also reads back the final cycle count and reports total rendering time in milliseconds.
Fixed-point conversion macros	Software Utility	Macros including <code>float2fix</code> , <code>fix2float</code> , <code>int2fix</code> , <code>fix2int</code> ,

9.3 Verilog

9.3.1 Design

Table 4: Program Listing Table: Verilog Lorenz System Design

Module	Type	Description
<code>testbench_lorenz</code>	Testbench	Top-level simulation harness. Generates <code>clk_50</code> and <code>clk_25</code> , applies reset sequencing, initializes fixed-point parameters and initial conditions, and instantiates <code>lorenz_system</code> as the DUT.
<code>integrator</code>	RTL (Sequential)	Single-state Euler accumulator used as an integrator. On reset, loads <code>InitialOut</code> into the internal register; otherwise updates state via <code>v1new = v1 + funct</code> . Outputs the current state <code>out = v1</code> .
<code>signed_mult</code>	RTL (Combinational)	Signed fixed-point multiplier for 7.20 two's-complement format. Forms the full product <code>mult_out = a*b</code> (54-bit) and truncates/selects bits to return a 27-bit 7.20 result via <code>{mult_out[53], mult_out[45:20]}</code> .
<code>lorenz_system</code>	RTL (Datapath + State)	Implements the Lorenz system Euler update in 7.20 fixed-point. Computes $\dot{x} = \sigma(y-x)$, $\dot{y} = x(\rho-z)-y$, $\dot{z} = xy-\beta z$ using shifts and <code>signed_mult</code> , then integrates <code>xk</code> , <code>yk</code> , <code>zk</code> using three <code>integrator</code> instances.

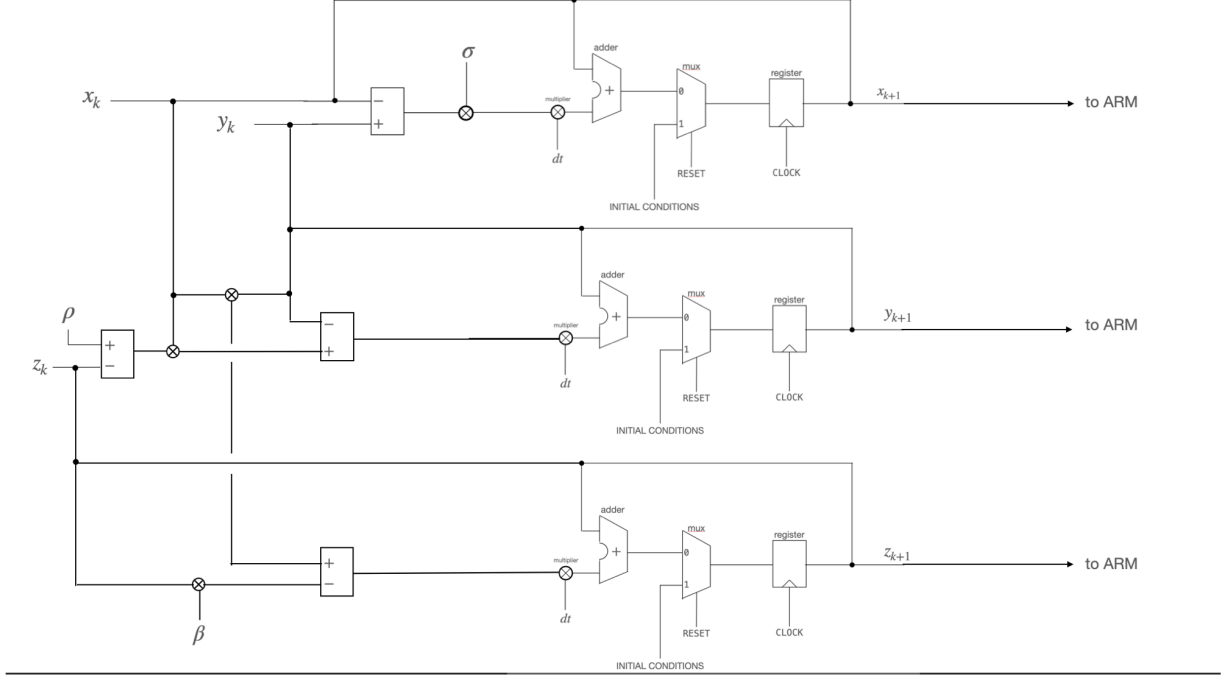


Figure 8: Overall hardware architecture of the Lorenz system implementation (adapted from [1]).

9.3.2 Platform Designer

We used Platform Designer (formerly QSys) to create the PIO connections on the Avalon bus that the HPS and FPGA use to communicate. We added a total of 10 PIO modules to this bus, 7 outputs and 3 inputs relative to the HPS. The directionality and bitwidths of these PIOs is described in Figure 9. The names are self descriptive, except for *HPS_CTRL*. This 8-bit PIO is used to drive the clock/integrator step (*HPS_CTRL*[0]) and *RESET* (*HPS_CTRL*[1]) from the HPS.

The parameters and X , Y , Z are represented in 7.20 fixed point notation on the FPGA side. Our C program has several macros defined for converting between fixed point and floating point for processing and display. These macros are summarized in Table 2. Because this is a 27 bit data type and C integers are 32 bits, values which were passed from the FPGA to the HPS had to be sign-extended to maintain their actual value in two's complement. This is done by pre-pending 5 copies of the MSB of each value on the FPGA side. The notation looks like the following:

```
.lorenz_x_external_connection_export ({5xk[26]}}, xk})
```

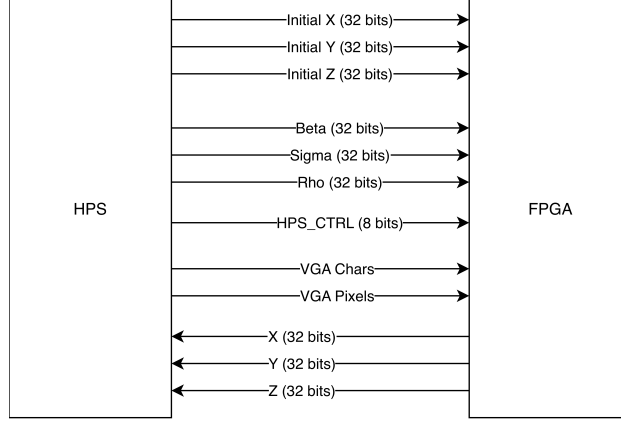


Figure 9: Directionality of PIO communication set in Platform Designer (QSys)

Platform Designer was also configured to allow the HPS to directly write the VGA pixel and char memory on the Avalon bus using the VGA Subsystem IP provided by Altera. This allowed us to draw pixels and characters on the VGA screen from the HPS directly. This was advantageous because it made it simple to interact with the VGA from our C program. In future labs, this method may introduce a memory bottleneck as it used a large chunk of the address space on the Avalon Bus. This also required data to be passed from the FPGA to the HPS, and then from the HPS to the VGA rather than directly from the FPGA to the VGA.

10 Additional Information

10.1 Mathematical Considerations

The specific form of integrator used for this hardware ODE solver applied to the integration of the Lorenz system used first order Euler Integration, a discrete time integrator that follows the equation that $x_{n+1} = x_n + h \cdot f$, where f represents the equations $dx = \sigma(y - x)$, $dy = x(\rho - z) - y$, $dz = xy - \beta z$. However, the Lorenz system is a continuous system, so the Euler integrator is approximating how the system behaves between ticks. We must consider the error accumulated through this method of integration as the real slope changes continuously in the Lorenz system, but we model it with Euler integration as remaining constant for Δt . This leads to both local errors at each time step, and as the Lorenz system is chaotic, also global errors which accumulate over time. The second point of error occurs from the use of 7.20 fixed-point. With 7.20 fixed-point, the smallest value that can be represented would be 2^{-20} or 9.53674316e-7. As a result of this, most real numbers get rounded. This happens not just for updated values, but also initial conditions and parameters. For example, if $\beta = 8/3$, this would need to be approximated to 2.66666603, a difference of 0.000000063, what may look like a small difference, but given the Lorenz system being chaotic, where it exhibits extreme sensitivity based on difference in initial conditions, this difference makes for different calculations as what may be calculated in a floating-point represented simulation. These differences should be interpreted not as errors, but as mathematical differences that can be explained through numeric representation. As a result, it is difficult to verify the

correctness of the calculated Lorenz system through numerical inspection, but correctness should be verified instead through short-term quantitative agreement and by observing the long-term qualitative behavior of the Lorenz attractor.

10.2 How you implemented the DDA circuits:

A Digital Differential Analyzer (DDA) simulates integration via methods such as Euler integration to numerically solve differential equations. Mathematically, this integrator has the form $x_{n+1} = x_n + (dt)(dx)$. The basic DDA is implemented via an adder and state value register, which adds the previous integrated value and a new derivative multiplied by a specific time step. For our Lorenz integrator, we created three of these basic DDAs with an additional mux to reset the integrator if desired which provided the initial conditions to the integrator to restart calculating. Additionally, to calculate dx , dy , and dz , we defined a series of equations, where $dx = \sigma(y - x)$, $dy = x(\rho - z) - y$, $dz = xy - \beta z$. This, theoretically, multiplied by dt , which at $1/256$ was the equivalent of a right bit shift of 8 and then was added to the previous value. Practically; however, as we have limited bits and the DSP modules on the DE1-SoC could do a maximum of 27×27 multiplies, our x , y , and z values were stored in 7.20 fixed point representation. This number system, as with any number system has the potential to overflow. As a result of this overflow, rather than bit shifting after fully calculating dx , dy , and dz , it was imperative to take intermediate steps to ensure that the values were divided before overflowing from large numbers multiplying rather than doing the multiplication first than dividing a possible overflown value. These hardware flows were as follows: $dxdt = \sigma((y - x) \ggg 8)$, $dydt = (((x \ggg 8)(\rho - z)) - (y \ggg 8))$, $dzdt = ((x \ggg 8)y - \beta(z \ggg 8))$.

11 Comments

This lab helped us understand the difference between writing code that looks correct and building hardware that actually works after synthesis. Much of our time was spent debugging rather than implementing the Lorenz equations themselves. Many early issues came from mismatched wire widths, signed vs. unsigned values, and incorrect module connections. In several cases the design compiled but produced incorrect behavior, which required us to inspect waveforms in ModelSim and SignalTap instead of only reading the code.

One of the main challenges was handling overflow and scaling in the 7.20 fixed-point format. The Lorenz equations are straightforward in floating-point software, but hardware multipliers have limited range, so some intermediate calculations became too large. We solved this by shifting values earlier in the datapath instead of only at the end of the computation. This made us better understand how number representation directly affects hardware behavior, especially for a chaotic system like the Lorenz attractor where small numerical changes quickly grow.

Working with the HPS and FPGA together was also an important part of the lab. Debugging communication required verifying that the FPGA produced correct values, the HPS read the correct addresses, and the display interpreted the fixed-point values properly. A mistake in any one of these steps looked similar from the screen output, so we had to test

each part separately using serial prints, simulation, and hardware probes.

12 AI Usage

Artificial intelligence tools were used only as a supplemental aid during this assignment. We primarily used AI to assist with debugging and understanding error messages produced during compilation and synthesis. In particular, it helped us interpret syntax errors, identify mismatched wire widths, and reason about incorrect logic connections that prevented successful hardware compilation. AI was also used as a learning aid to clarify hardware description language concepts and to better understand why certain implementations did not behave as expected. Additionally, it was used for improving the clarity of writing in this report by helping reorganize our thoughts into concise, formal technical language and improving sentence structure and readability. All design decisions, Verilog/SystemVerilog implementation, testing, and verification of the hardware were completed by us. The AI tools were not used to generate the actual hardware design or to replace understanding, but rather to support debugging, error interpretation, and refinement of written explanations.

References

- [1] Van Hunter Adams, *ECE 5760: Hardware ODE Solver (Lorenz System) Lab*, Cornell University. Available: <https://vanhunteradams.com/DE1/Lorenz/Lorenz.html#Lab-reports> Accessed: February 2026.

12.1 Program Listings

Listing 1: C program for Lorenz control and display

```
1
2 ///////////////////////////////////////////////////
3 ///
4 /// compile with
5 /// gcc lorenz.c -o lz -O2 -lm
6 ///
7 ///////////////////////////////////////////////////
8 #include <stdio.h>
9 #include <stdlib.h>
10 #include <unistd.h>
11 #include <fcntl.h>
12 #include <sys/types.h>
13 #include <sys/ipc.h>
14 #include <sys/shm.h>
15 #include <sys/mman.h>
16 #include <sys/time.h>
17 #include <math.h>
18 #include <stdint.h>
```

```

19 #include <pthread.h>
20 #include <string.h>
21
22 // video display
23 #define SDRAM_BASE          0xC0000000
24 #define SDRAM_END          0xC3FFFFFF
25 #define SDRAM_SPAN         0x04000000
26 // characters
27 #define FPGA_CHAR_BASE     0xC9000000
28 #define FPGA_CHAR_END     0xC9001FFF
29 #define FPGA_CHAR_SPAN    0x00002000
30 /* Cyclone V FPGA devices */
31 #define HW_REGS_BASE       0xff200000
32 // #define HW_REGS_SPAN    0x00200000
33 #define HW_REGS_SPAN       0x00005000
34
35 // Lorenz constants
36 #define SCALE 1048576 // 2^20
37
38 // LW Bus Offsets
39 #define X_OFFSET 0x10
40 #define Y_OFFSET 0x20
41 #define Z_OFFSET 0x30
42 #define SIGMA_OFFSET 0x40
43 #define BETA_OFFSET 0x50
44 #define RHO_OFFSET 0x60
45 #define X_INIT_OFFSET 0x70
46 #define Y_INIT_OFFSET 0x80
47 #define Z_INIT_OFFSET 0x90
48 #define HPS_CTRL_OFFSET 0x100 // bit 0 for clk, bit 1 for reset, ...
49
50 // graphics primitives
51 void VGA_text (int, int, char *);
52 void VGA_text_clear();
53 void VGA_box (int, int, int, int, short);
54 void VGA_rect (int, int, int, int, short);
55 void VGA_line(int, int, int, int, short) ;
56 void VGA_Vline(int, int, int, short) ;
57 void VGA_Hline(int, int, int, short) ;
58 void VGA_disc (int, int, int, short);
59 void VGA_circle (int, int, int, int);
60 // 16-bit primary colors
61 #define red (0+(0<<5)+(31<<11))
62 #define dark_red (0+(0<<5)+(15<<11))
63 #define green (0+(63<<5)+(0<<11))
64 #define dark_green (0+(31<<5)+(0<<11))
65 #define blue (31+(0<<5)+(0<<11))

```

```

66 #define dark_blue (15+(0<<5)+(0<<11))
67 #define yellow (0+(63<<5)+(31<<11))
68 #define cyan (31+(63<<5)+(0<<11))
69 #define magenta (31+(0<<5)+(31<<11))
70 #define black (0x0000)
71 #define gray (15+(31<<5)+(51<<11))
72 #define white (0xffff)
73 int colors[] = {red, dark_red, green, dark_green, blue, dark_blue,
74               yellow, cyan, magenta, gray, black, white};
75
76 // pixel macro
77 #define VGA_PIXEL(x,y,color) do{\
78     int *pixel_ptr ;\
79     pixel_ptr = (int*)((char *)vga_pixel_ptr + (((y)*640+(x))<<1))
80     ; \
81     *(short *)pixel_ptr = (color);\
82 } while(0)
83
84 // the light weight buss base
85 void *h2p_lw_virtual_base;
86
87 // AXI Bus Pointers
88 volatile int *lorenz_x_ptr = NULL;
89 volatile int *lorenz_y_ptr = NULL;
90 volatile int *lorenz_z_ptr = NULL;
91 volatile int *sigma_ptr = NULL;
92 volatile int *beta_ptr = NULL;
93 volatile int *rho_ptr = NULL;
94 volatile int *x_init_ptr = NULL;
95 volatile int *y_init_ptr = NULL;
96 volatile int *z_init_ptr = NULL;
97 volatile uint8_t *hps_ctrl_ptr = NULL; // Bit 0: Clock | Bit 1:
98 Reset
99
100 // pixel buffer
101 volatile unsigned int * vga_pixel_ptr = NULL ;
102 void *vga_pixel_virtual_base;
103
104 // character buffer
105 volatile unsigned int * vga_char_ptr = NULL ;
106 void *vga_char_virtual_base;
107
108 // /dev/mem file id
109 int fd;
110
111 // measure time
112 struct timeval t1, t2;

```

```

111 double elapsedTime;
112
113 #define RESET_HIGH *hps_ctrl_ptr = *hps_ctrl_ptr | 0b00000010; //
    Mask reset HIGH
114 #define RESET_LOW *hps_ctrl_ptr = *hps_ctrl_ptr & 0b11111101; //
    Mask reset LOW
115 #define CLK_HIGH *hps_ctrl_ptr = *hps_ctrl_ptr | 0b00000001; //
    Mask clock HIGH
116 #define CLK_LOW *hps_ctrl_ptr = *hps_ctrl_ptr & 0b11111110; //
    Mask clock LOW
117
118 #define float2fix(a) ((signed int)(a * (SCALE)))
119 #define fix2float(a) ((float)(a) / (SCALE))
120 #define int2fix(a) ((signed int)(a << 20))
121 #define fix2int(a) ((int)((a >> 20)))
122
123 int running = 1;
124 int reset = 0;
125 volatile int speed = 1000;
126
127 char sigma_text[20];
128 char beta_text[20];
129 char rho_text[20];
130 char x_init_text[20];
131 char y_init_text[20];
132 char z_init_text[20];
133
134 void * graphics_thread()
135 {
136     while(1)
137     {
138         if (reset)
139         {
140             reset = 0;
141             VGA_box(0,0,639,479, black); // Clear the screen on
                reset
142         }
143
144         if (running) // Toggle clock
145         {
146             CLK_HIGH;
147             usleep(speed);
148             CLK_LOW;
149         }
150
151         // xy plot
152         VGA_rect(40, 20, 260, 240, white);

```

```

153     VGA_text(5, 1, "XY");
154
155     // xz plot
156     VGA_rect(300, 20, 520, 240, white);
157     VGA_text(37, 1, "XZ");
158
159     // yz plot
160     VGA_rect(170, 258, 390, 478, white);
161     VGA_text(22, 31, "YZ");
162
163     // xy center - (150, 140)
164     VGA_PIXEL((*lorenz_x_ptr >> 18) + 150, (-(*lorenz_y_ptr >>
165         18) + 125), magenta);
166
167     // xz origin - (410, 60)
168     VGA_PIXEL((*lorenz_x_ptr >> 18) + 420, (-(*lorenz_z_ptr >>
169         18) + 240), yellow);
170
171     // yz origin
172     VGA_PIXEL((*lorenz_y_ptr >> 18) + 282, (-(*lorenz_z_ptr >>
173         18) + 470), cyan);
174
175     sprintf(&sigma_text, "sigma: %f", fix2float(*sigma_ptr));
176     sprintf(&beta_text, "beta: %f", fix2float(*beta_ptr));
177     sprintf(&rho_text, "rho: %f", fix2float(*rho_ptr));
178     sprintf(&x_init_text, "x_init: %f", fix2float(*x_init_ptr));
179     sprintf(&y_init_text, "y_init: %f", fix2float(*y_init_ptr));
180     sprintf(&z_init_text, "z_init: %f", fix2float(*z_init_ptr));
181
182     VGA_text(50, 35, "Parameters");
183     VGA_text(50, 37, sigma_text);
184     VGA_text(50, 39, beta_text);
185     VGA_text(50, 41, rho_text);
186     VGA_text(50, 43, x_init_text);
187     VGA_text(50, 45, y_init_text);
188     VGA_text(50, 47, z_init_text);
189     VGA_text(0, 43, "Md Shad");
190     VGA_text(0, 45, "Michael Grillo");
191     VGA_text(0, 47, "Stephen Barlett");
192     // VGA_text(0, 53, "Jamayne (SpaceX)");
193
194     } // end while(1)
195
196 void * serial_thread()

```

```

197 {
198     char input[50];
199
200     while(1)
201     {
202         printf("Enter character
203                r/b/s/x/y/z/start/stop/reset/[(speed up)](slow down):
204                ");
205
206         scanf("%s", input);
207         if (strcmp(input, "reset") == 0)
208         {
209             reset = 1;
210             RESET_HIGH;
211             CLK_LOW;
212             RESET_LOW;
213             CLK_HIGH;
214             CLK_LOW;
215             RESET_HIGH;
216             printf("System reset\n");
217         }
218         else if (strcmp(input, "start") == 0)
219         {
220             running = 1;
221             printf("Computation started\n");
222         }
223         else if (strcmp(input, "stop") == 0)
224         {
225             running = 0;
226             printf("Computation stopped\n");
227         }
228         else if (strcmp(input, "s") == 0)
229         {
230             float new_sigma = 0;
231             printf("Enter new sigma: ");
232             scanf("%f", &new_sigma);
233             *sigma_ptr = (signed int)((new_sigma) * SCALE);
234             printf("New sigma is: %f", new_sigma);
235         }
236         // Used AI to produce similar code blocks to save time
237         ##### BEGIN AI-GENERATED CODE
238         #####
239         else if (strcmp(input, "x") == 0)
240         {
241             float new_x_init = 0;
242             printf("Enter new x_init: ");
243             scanf("%f", &new_x_init);

```

```

241         *x_init_ptr = (signed int)((new_x_init) * SCALE);
242         printf("New x_init is: %f", new_x_init);
243     }
244     else if (strcmp(input, "y") == 0)
245     {
246         float new_y_init = 0;
247         printf("Enter new y_init: ");
248         scanf("%f", &new_y_init);
249         *y_init_ptr = (signed int)((new_y_init) * SCALE);
250         printf("New y_init is: %f", new_y_init);
251     }
252     else if (strcmp(input, "z") == 0)
253     {
254         float new_z_init = 0;
255         printf("Enter new z_init: ");
256         scanf("%f", &new_z_init);
257         *z_init_ptr = (signed int)((new_z_init) * SCALE);
258         printf("New z_init is: %f", new_z_init);
259     }
260     ##### END AI-GENERATED CODE
261     #####
262     else if (strcmp(input, "r") == 0)
263     {
264         float new_rho = 0;
265         printf("Enter new rho: ");
266         scanf("%f", &new_rho);
267         *rho_ptr = (signed int)((new_rho) * SCALE);
268         printf("New rho is: %f", new_rho);
269     }
270     else if (strcmp(input, "b") == 0)
271     {
272         float new_beta = 0;
273         printf("Enter new beta: ");
274         scanf("%f", &new_beta);
275         *beta_ptr = (signed int)((new_beta) * SCALE);
276         printf("New beta is: %f", new_beta);
277     }
278     else if (strcmp(input, "[" ) == 0)
279     {
280         if (speed > 100)
281         {
282             printf("Decreasing speed delay");
283             speed -= 100;
284         }
285     }
286     else if (strcmp(input, "]" ) == 0)
287     {

```

```

287         if (speed < 10000)
288         {
289             printf("Increasing speed delay");
290             speed += 100;
291         }
292     }
293     else
294     {
295         printf("Invalid input\n");
296     }
297 }
298 }
299
300 int main(void)
301 {
302
303     // === need to mmap: =====
304     // FPGA_CHAR_BASE
305     // FPGA_ONCHIP_BASE
306     // HW_REGS_BASE
307
308     // === get FPGA addresses =====
309     // Open /dev/mem
310     if( ( fd = open( "/dev/mem", ( O_RDWR | O_SYNC ) ) ) == -1 )
311     {
312         printf( "ERROR: could not open \"/dev/mem\"...\n" );
313         return( 1 );
314     }
315
316     // get virtual addr that maps to physical
317     h2p_lw_virtual_base = mmap( NULL, HW_REGS_SPAN, ( PROT_READ |
318         PROT_WRITE ), MAP_SHARED, fd, HW_REGS_BASE );
319     if( h2p_lw_virtual_base == MAP_FAILED ) {
320         printf( "ERROR: mmap1() failed...\n" );
321         close( fd );
322         return(1);
323     }
324
325     lorenz_x_ptr = (volatile int *)(h2p_lw_virtual_base + X_OFFSET);
326     lorenz_y_ptr = (volatile int *)(h2p_lw_virtual_base + Y_OFFSET);
327     lorenz_z_ptr = (volatile int *)(h2p_lw_virtual_base + Z_OFFSET);
328     sigma_ptr = (volatile int *)(h2p_lw_virtual_base +
329         SIGMA_OFFSET);
330     beta_ptr = (volatile int *)(h2p_lw_virtual_base + BETA_OFFSET);
331     rho_ptr = (volatile int *)(h2p_lw_virtual_base + RHO_OFFSET);
332     x_init_ptr = (volatile int *)(h2p_lw_virtual_base +
333         X_INIT_OFFSET);

```

```

330 y_init_ptr = (volatile int *) (h2p_lw_virtual_base +
    Y_INIT_OFFSET);
331 z_init_ptr = (volatile int *) (h2p_lw_virtual_base +
    Z_INIT_OFFSET);
332 hps_ctrl_ptr = (volatile uint8_t *) (h2p_lw_virtual_base +
    HPS_CTRL_OFFSET);
333
334 // === get VGA char addr =====
335 // get virtual addr that maps to physical
336 vga_char_virtual_base = mmap( NULL, FPGA_CHAR_SPAN, (
    PROT_READ | PROT_WRITE ), MAP_SHARED, fd, FPGA_CHAR_BASE );
337 if( vga_char_virtual_base == MAP_FAILED ) {
338     printf( "ERROR: mmap2() failed...\n" );
339     close( fd );
340     return(1);
341 }
342
343 // Get the address that maps to the FPGA LED control
344 vga_char_ptr =(unsigned int *) (vga_char_virtual_base);
345
346 // === get VGA pixel addr =====
347 // get virtual addr that maps to physical
348 vga_pixel_virtual_base = mmap( NULL, SDRAM_SPAN, ( PROT_READ |
    PROT_WRITE ), MAP_SHARED, fd, SDRAM_BASE);
349 if( vga_pixel_virtual_base == MAP_FAILED ) {
350     printf( "ERROR: mmap3() failed...\n" );
351     close( fd );
352     return(1);
353 }
354
355 // Get the address that maps to the FPGA pixel buffer
356 vga_pixel_ptr =(unsigned int *) (vga_pixel_virtual_base);
357
358 // =====
359
360 /* create a message to be displayed on the VGA
    and LCD displays */
361 char text_top_row[40] = "DE1-SoC ARM/FPGA\0";
362 char text_bottom_row[40] = "Cornell ece5760\0";
363 char text_next[40] = "Graphics primitives\0";
364 char num_string[20], time_string[20] ;
365 char color_index = 0 ;
366 int color_counter = 0 ;
367
368
369 // position of disk primitive
370 int disc_x = 0;
371 // position of circle primitive

```

```

372     int circle_x = 0 ;
373     // position of box primitive
374     int box_x = 5 ;
375     // position of vertical line primitive
376     int Vline_x = 350;
377     // position of horizontal line primitive
378     int Hline_y = 250;
379
380     //VGA_text (34, 1, text_top_row);
381     //VGA_text (34, 2, text_bottom_row);
382     // clear the screen
383     VGA_box (0, 0, 639, 479, 0x0000);
384     // clear the text
385     VGA_text_clear();
386     // write text
387
388     // R bits 11-15 mask 0xf800
389     // G bits 5-10  mask 0x07e0
390     // B bits 0-4   mask 0x001f
391     // so color = B+(G<<5)+(R<<11);
392
393     *x_init_ptr = SCALE * -1;
394     *y_init_ptr = SCALE / 10;
395     *z_init_ptr = SCALE * 25;
396     *sigma_ptr = SCALE * 10;
397     *beta_ptr = SCALE * (8.0 / 3.0);
398     *rho_ptr = SCALE * 28;
399     printf("Initial conditions: x: %f, y: %f, z: %f \n",
400           fix2float(*x_init_ptr), fix2float(*y_init_ptr),
401           fix2float(*z_init_ptr));
402     printf("Initial conditions: r: %f, b: %f, s: %f \n",
403           fix2float(*sigma_ptr), fix2float(*beta_ptr),
404           fix2float(*rho_ptr));
405
406     // Toggle reset to start integrator
407     RESET_HIGH;
408     CLK_LOW;
409     RESET_LOW;
410     CLK_HIGH;
411     CLK_LOW;
412     RESET_HIGH;
413
414     pthread_t thread_graphics, thread_serial;
415
416     pthread_create(&thread_serial, NULL, serial_thread, NULL);
417     pthread_create(&thread_graphics, NULL, graphics_thread, NULL);

```

```

415     pthread_join(thread_serial, NULL);
416     pthread_join(thread_graphics, NULL);
417
418 } // end main
419
420 /*****
421  * Subroutine to send a string of text to the VGA monitor
422  *****/
423 void VGA_text(int x, int y, char * text_ptr)
424 {
425     volatile char * character_buffer = (char *) vga_char_ptr ; //
426     VGA character buffer
427     int offset;
428     /* assume that the text string fits on one line */
429     offset = (y << 7) + x;
430     while ( *(text_ptr) )
431     {
432         // write to the character buffer
433         *(character_buffer + offset) = *(text_ptr);
434         ++text_ptr;
435         ++offset;
436     }
437 }
438
439 /*****
440  * Subroutine to clear text to the VGA monitor
441  *****/
442 void VGA_text_clear()
443 {
444     volatile char * character_buffer = (char *) vga_char_ptr ; //
445     VGA character buffer
446     int offset, x, y;
447     for (x=0; x<79; x++){
448         for (y=0; y<59; y++){
449             /* assume that the text string fits on one line */
450             offset = (y << 7) + x;
451             // write to the character buffer
452             *(character_buffer + offset) = ' ';
453         }
454     }
455 }
456
457 /*****
458  * Draw a filled rectangle on the VGA monitor
459  *****/
460 #define SWAP(X,Y) do{int temp=X; X=Y; Y=temp;}while(0)

```

```

460 void VGA_box(int x1, int y1, int x2, int y2, short pixel_color)
461 {
462     char *pixel_ptr ;
463     int row, col;
464
465     /* check and fix box coordinates to be valid */
466     if (x1>639) x1 = 639;
467     if (y1>479) y1 = 479;
468     if (x2>639) x2 = 639;
469     if (y2>479) y2 = 479;
470     if (x1<0) x1 = 0;
471     if (y1<0) y1 = 0;
472     if (x2<0) x2 = 0;
473     if (y2<0) y2 = 0;
474     if (x1>x2) SWAP(x1,x2);
475     if (y1>y2) SWAP(y1,y2);
476     for (row = y1; row <= y2; row++)
477         for (col = x1; col <= x2; ++col)
478             {
479                 //640x480
480                 //pixel_ptr = (char *)vga_pixel_ptr + (row<<10) +
481                     col ;
482                 // set pixel color
483                 /*(char *)pixel_ptr = pixel_color;
484                 VGA_PIXEL(col,row,pixel_color);
485             }
486
487 /*****
488  * Draw a outline rectangle on the VGA monitor
489  *****/
490 #define SWAP(X,Y) do{int temp=X; X=Y; Y=temp;}while(0)
491
492 void VGA_rect(int x1, int y1, int x2, int y2, short pixel_color)
493 {
494     char *pixel_ptr ;
495     int row, col;
496
497     /* check and fix box coordinates to be valid */
498     if (x1>639) x1 = 639;
499     if (y1>479) y1 = 479;
500     if (x2>639) x2 = 639;
501     if (y2>479) y2 = 479;
502     if (x1<0) x1 = 0;
503     if (y1<0) y1 = 0;
504     if (x2<0) x2 = 0;
505     if (y2<0) y2 = 0;

```

```

506     if (x1>x2) SWAP(x1,x2);
507     if (y1>y2) SWAP(y1,y2);
508     // left edge
509     col = x1;
510     for (row = y1; row <= y2; row++){
511         //640x480
512         //pixel_ptr = (char *)vga_pixel_ptr + (row<<10) + col ;
513         // set pixel color
514         /*(char *)pixel_ptr = pixel_color;
515         VGA_PIXEL(col,row,pixel_color);
516     }
517
518     // right edge
519     col = x2;
520     for (row = y1; row <= y2; row++){
521         //640x480
522         //pixel_ptr = (char *)vga_pixel_ptr + (row<<10) + col ;
523         // set pixel color
524         /*(char *)pixel_ptr = pixel_color;
525         VGA_PIXEL(col,row,pixel_color);
526     }
527
528     // top edge
529     row = y1;
530     for (col = x1; col <= x2; ++col){
531         //640x480
532         //pixel_ptr = (char *)vga_pixel_ptr + (row<<10) + col ;
533         // set pixel color
534         /*(char *)pixel_ptr = pixel_color;
535         VGA_PIXEL(col,row,pixel_color);
536     }
537
538     // bottom edge
539     row = y2;
540     for (col = x1; col <= x2; ++col){
541         //640x480
542         //pixel_ptr = (char *)vga_pixel_ptr + (row<<10) + col ;
543         // set pixel color
544         /*(char *)pixel_ptr = pixel_color;
545         VGA_PIXEL(col,row,pixel_color);
546     }
547 }
548
549 /*****
550  * Draw a horixontal line on the VGA monitor
551  *****/
552 #define SWAP(X,Y) do{int temp=X; X=Y; Y=temp;}while(0)

```

```

553
554 void VGA_Hline(int x1, int y1, int x2, short pixel_color)
555 {
556     char *pixel_ptr ;
557     int row, col;
558
559     /* check and fix box coordinates to be valid */
560     if (x1>639) x1 = 639;
561     if (y1>479) y1 = 479;
562     if (x2>639) x2 = 639;
563     if (x1<0) x1 = 0;
564     if (y1<0) y1 = 0;
565     if (x2<0) x2 = 0;
566     if (x1>x2) SWAP(x1,x2);
567     // line
568     row = y1;
569     for (col = x1; col <= x2; ++col){
570         //640x480
571         //pixel_ptr = (char *)vga_pixel_ptr + (row<<10) + col ;
572         // set pixel color
573         /*(char *)pixel_ptr = pixel_color;
574         VGA_PIXEL(col,row,pixel_color);
575     }
576 }
577
578 /*****
579  * Draw a vertical line on the VGA monitor
580 *****/
581 #define SWAP(X,Y) do{int temp=X; X=Y; Y=temp;}while(0)
582
583 void VGA_Vline(int x1, int y1, int y2, short pixel_color)
584 {
585     char *pixel_ptr ;
586     int row, col;
587
588     /* check and fix box coordinates to be valid */
589     if (x1>639) x1 = 639;
590     if (y1>479) y1 = 479;
591     if (y2>479) y2 = 479;
592     if (x1<0) x1 = 0;
593     if (y1<0) y1 = 0;
594     if (y2<0) y2 = 0;
595     if (y1>y2) SWAP(y1,y2);
596     // line
597     col = x1;
598     for (row = y1; row <= y2; row++){
599         //640x480

```

```

600     //pixel_ptr = (char *)vga_pixel_ptr + (row<<10) + col ;
601     // set pixel color
602     /*(char *)pixel_ptr = pixel_color;
603     VGA_PIXEL(col,row,pixel_color);
604 }
605 }
606
607
608 /*****
609  * Draw a filled circle on the VGA monitor
610  *****/
611
612 void VGA_disc(int x, int y, int r, short pixel_color)
613 {
614     char *pixel_ptr ;
615     int row, col, rsqr, xc, yc;
616
617     rsqr = r*r;
618
619     for (yc = -r; yc <= r; yc++)
620         for (xc = -r; xc <= r; xc++)
621         {
622             col = xc;
623             row = yc;
624             // add the r to make the edge smoother
625             if(col*col+row*row <= rsqr+r){
626                 col += x; // add the center point
627                 row += y; // add the center point
628                 //check for valid 640x480
629                 if (col>639) col = 639;
630                 if (row>479) row = 479;
631                 if (col<0) col = 0;
632                 if (row<0) row = 0;
633                 //pixel_ptr = (char *)vga_pixel_ptr + (row<<10) +
634                     col ;
635                 // set pixel color
636                 /*(char *)pixel_ptr = pixel_color;
637                 VGA_PIXEL(col,row,pixel_color);
638             }
639         }
640     }
641 }
642
643 /*****
644  * Draw a circle on the VGA monitor
645  *****/

```

```

646 void VGA_circle(int x, int y, int r, int pixel_color)
647 {
648     char *pixel_ptr ;
649     int row, col, rsqr, xc, yc;
650     int col1, row1;
651     rsqr = r*r;
652
653     for (yc = -r; yc <= r; yc++){
654         //row = yc;
655         col1 = (int)sqrt((float)(rsqr + r - yc*yc));
656         // right edge
657         col = col1 + x; // add the center point
658         row = yc + y; // add the center point
659         //check for valid 640x480
660         if (col>639) col = 639;
661         if (row>479) row = 479;
662         if (col<0) col = 0;
663         if (row<0) row = 0;
664         //pixel_ptr = (char *)vga_pixel_ptr + (row<<10) + col ;
665         // set pixel color
666         /*(char *)pixel_ptr = pixel_color;
667         VGA_PIXEL(col,row,pixel_color);
668         // left edge
669         col = -col1 + x; // add the center point
670         //check for valid 640x480
671         if (col>639) col = 639;
672         if (row>479) row = 479;
673         if (col<0) col = 0;
674         if (row<0) row = 0;
675         //pixel_ptr = (char *)vga_pixel_ptr + (row<<10) + col ;
676         // set pixel color
677         /*(char *)pixel_ptr = pixel_color;
678         VGA_PIXEL(col,row,pixel_color);
679     }
680     for (xc = -r; xc <= r; xc++){
681         //row = yc;
682         row1 = (int)sqrt((float)(rsqr + r - xc*xc));
683         // right edge
684         col = xc + x; // add the center point
685         row = row1 + y; // add the center point
686         //check for valid 640x480
687         if (col>639) col = 639;
688         if (row>479) row = 479;
689         if (col<0) col = 0;
690         if (row<0) row = 0;
691         //pixel_ptr = (char *)vga_pixel_ptr + (row<<10) + col ;
692         // set pixel color

```

```

693     /*(char *)pixel_ptr = pixel_color;
694     VGA_PIXEL(col,row,pixel_color);
695     // left edge
696     row = -row1 + y; // add the center point
697     //check for valid 640x480
698     if (col>639) col = 639;
699     if (row>479) row = 479;
700     if (col<0) col = 0;
701     if (row<0) row = 0;
702     //pixel_ptr = (char *)vga_pixel_ptr + (row<<10) + col ;
703     // set pixel color
704     /*(char *)pixel_ptr = pixel_color;
705     VGA_PIXEL(col,row,pixel_color);
706 }
707 }
708
709 // =====
710 // === Draw a line
711 // =====
712 //plot a line
713 //at x1,y1 to x2,y2 with color
714 //Code is from David Rodgers,
715 //"Procedural Elements of Computer Graphics",1985
716 void VGA_line(int x1, int y1, int x2, int y2, short c) {
717     int e;
718     signed int dx,dy,j, temp;
719     signed int s1,s2, xchange;
720     signed int x,y;
721     char *pixel_ptr ;
722
723     /* check and fix line coordinates to be valid */
724     if (x1>639) x1 = 639;
725     if (y1>479) y1 = 479;
726     if (x2>639) x2 = 639;
727     if (y2>479) y2 = 479;
728     if (x1<0) x1 = 0;
729     if (y1<0) y1 = 0;
730     if (x2<0) x2 = 0;
731     if (y2<0) y2 = 0;
732
733     x = x1;
734     y = y1;
735
736     //take absolute value
737     if (x2 < x1) {
738         dx = x1 - x2;
739         s1 = -1;

```

```

740     }
741
742     else if (x2 == x1) {
743         dx = 0;
744         s1 = 0;
745     }
746
747     else {
748         dx = x2 - x1;
749         s1 = 1;
750     }
751
752     if (y2 < y1) {
753         dy = y1 - y2;
754         s2 = -1;
755     }
756
757     else if (y2 == y1) {
758         dy = 0;
759         s2 = 0;
760     }
761
762     else {
763         dy = y2 - y1;
764         s2 = 1;
765     }
766
767     xchange = 0;
768
769     if (dy>dx) {
770         temp = dx;
771         dx = dy;
772         dy = temp;
773         xchange = 1;
774     }
775
776     e = ((int)dy<<1) - dx;
777
778     for (j=0; j<=dx; j++) {
779         //video_pt(x,y,c); //640x480
780         //pixel_ptr = (char *)vga_pixel_ptr + (y<<10)+ x;
781         // set pixel color
782         /*(char *)pixel_ptr = c;
783         VGA_PIXEL(x,y,c);
784
785         if (e>=0) {
786             if (xchange==1) x = x + s1;

```

```

787         else y = y + s2;
788         e = e - ((int)dx<<1);
789     }
790
791     if (xchange==1) y = y + s2;
792     else x = x + s1;
793
794     e = e + ((int)dy<<1);
795 }
796 }

```

Listing 2: Verilog for Lorenz integrator and simulator test bench.

```

1  `timescale 1ns/1ns
2
3  module testbench_lorenz();
4      localparam integer SCALE = 1048576; // 2^20, needed for using
5          7:20 Fixed Point
6
7      reg clk_50, clk_25, reset;
8
9      reg signed [26:0] xk_initial;
10     reg signed [26:0] yk_initial;
11     reg signed [26:0] zk_initial;
12
13     reg signed [26:0] sigma;
14     reg signed [26:0] beta;
15     reg signed [26:0] rho;
16     reg signed [26:0] dt;
17
18     wire signed [26:0] xk;
19     wire signed [26:0] yk;
20     wire signed [26:0] zk;
21
22     // Instantiate Device Under Test
23     lorenz_system DUT (
24         .clk(clk_50),
25         .reset(reset),
26         .sigma(sigma),
27         .beta(beta),
28         .rho(rho),
29         .dt(dt),
30         .xk_initial(xk_initial),
31         .yk_initial(yk_initial),
32         .zk_initial(zk_initial),
33         .xk(xk),
34         .yk(yk),
35         .zk(zk)

```

```

35 );
36
37 // Initialize Parameters
38 initial begin
39     clk_50 = 1'b0;
40     clk_25 = 1'b0;
41     xk_initial = -1 * SCALE;
42     yk_initial = SCALE / 10;
43     zk_initial = 25 * SCALE;
44     sigma = 10 * SCALE;
45     beta = 8 * SCALE / 3;
46     rho = 28 * SCALE;
47     dt = SCALE / 256;
48
49 end
50
51 // Reset system
52 initial begin
53     reset = 1'b1;
54     #10
55     reset = 1'b0;
56     #30
57     reset = 1'b1;
58 end
59
60 //Toggle the clocks
61 always begin
62     #10
63     clk_50 = !clk_50;
64 end
65
66 always begin
67     #20
68     clk_25 = !clk_25;
69 end
70
71 endmodule
72
73 //////////////////////////////////////
74 //// integrator //////////////////////////////////////
75 //////////////////////////////////////
76
77 module integrator(out,funct,InitialOut,clk,reset);
78     output signed [26:0] out;          //the state variable V
79     input signed [26:0] funct;         //the dV/dt function
80     input clk, reset;
81     input signed [26:0] InitialOut;    //the initial state variable V

```

```

82
83     wire signed [26:0] out, v1new ;
84     reg signed [26:0] v1 ;
85
86     always @ (posedge clk) begin
87
88         if (reset==1) //reset
89             v1 <= InitialOut ;
90         else
91             v1 <= v1new ;
92         end
93
94         assign v1new = v1 + funct ;
95         assign out = v1 ;
96     endmodule
97
98     //////////////////////////////////////
99     ///// signed mult of 7.20 format 2'comp////////
100    //////////////////////////////////////
101
102    module signed_mult (out, a, b);
103        output signed [26:0] out;
104        input signed [26:0] a;
105        input signed [26:0] b;
106        // intermediate full bit length
107        wire signed [53:0] mult_out;
108        assign mult_out = a * b;
109        // select bits for 7.20 fixed point
110        assign out = {mult_out[53], mult_out[45:20]};
111    endmodule
112    //////////////////////////////////////
113
114
115    //////////////////////////////////////
116    // Module for Lorenz System Euler Integrator
117    //////////////////////////////////////
118    module lorenz_system (
119        input wire clk,
120        input wire reset,
121
122        input wire signed [26:0] sigma,
123        input wire signed [26:0] beta,
124        input wire signed [26:0] rho,
125        input wire signed [26:0] dt,
126
127        input wire signed [26:0] xk_initial,
128        input wire signed [26:0] yk_initial,

```

```

129     input wire signed [26:0] zk_initial,
130
131     output wire signed [26:0] xk,
132     output wire signed [26:0] yk,
133     output wire signed [26:0] zk
134
135 );
136 // intermediate wires for xk
137 wire signed [26:0] y_sub_x;
138 wire signed [26:0] y_sub_x_shift;
139 //wire signed [26:0] dx;
140 wire signed [26:0] dxdt;
141 // intermediate wires for yk
142 wire signed [26:0] rho_sub_z;
143 wire signed [26:0] x_shift;
144 wire signed [26:0] y_shift;
145 wire signed [26:0] z_shift;
146 wire signed [26:0] rho_sub_z_times_x;
147 //wire signed [26:0] dy;
148 wire signed [26:0] dydt;
149 // intermediate wires for zk
150 wire signed [26:0] x_times_y;
151 wire signed [26:0] beta_times_z;
152 wire signed [26:0] beta_times_z_shift;
153 //wire signed [26:0] dz;
154 wire signed [26:0] dzdt;
155
156 // iteration 2: updated to match recommended style to prvenet
    overflow
157 assign x_shift = xk >>> 8;
158 assign y_shift = yk >>> 8;
159 assign z_shift = zk >>> 8;
160
161 // calculate dx
162 assign y_sub_x = yk - xk;
163 assign y_sub_x_shift = y_sub_x >>> 8;
164 signed_mult ysxts(.out(dxdt), .a(sigma), .b(y_sub_x_shift));
165
166 // calculate dy
167 assign rho_sub_z = rho - zk;
168 signed_mult rszyx(.out(rho_sub_z_times_x), .a(rho_sub_z),
    .b(x_shift));
169
170 assign dydt = rho_sub_z_times_x - y_shift;
171
172
173 // calculate dz

```

```

174 signed_mult xty(.out(x_times_y), .a(x_shift), .b(yk));
175 signed_mult btz(.out(beta_times_z), .a(beta), .b(z_shift));
176 assign dzdt = x_times_y - beta_times_z;
177
178 // Apply integrator function
179 integrator xint (.out(xk), .funct(dxdt), .InitialOut(xk_initial),
    .clk(clk), .reset(reset));
180 integrator yint (.out(yk), .funct(dydt), .InitialOut(yk_initial),
    .clk(clk), .reset(reset));
181 integrator zint (.out(zk), .funct(dzdt), .InitialOut(zk_initial),
    .clk(clk), .reset(reset));
182
183 endmodule

```

AI Prompt Interaction Log

This appendix documents the use of an AI coding assistant during development of the HPS control interface for the Lorenz system solver. The log summarizes prompts, suggested edits, and code changes produced during the interaction. Note that acceptance of changes cannot be directly measured; counts reflect suggestions provided by the assistant.

Summary Table

#	Date	Time (EST)	Prompt Summary	Edits	Insertions	Mods	Dels
1	Feb 13, 2026	10:06 AM	Rewrite C code that updates the x initial condition so it also supports y and z state variables for the Lorenz solver.	3	2	1	0

Original User Code

```

else if (strcmp(input, "x") == 0)
{
    float new_x_init = 0;
    printf("Enter new x_init: ");
    scanf("%f", &new_x_init);
    *x_init_ptr = (signed int)((new_x_init) * SCALE);
}

```

```
    printf("New x_init is: %f", new_x_init);
}
```

Assistant Suggested Changes

The assistant recommended generalizing the handler so the terminal interface can modify all three Lorenz system state variables. The following conceptual changes were suggested:

- Rename variable: `new_x_init` $\rightarrow$ `new_y_init`, `new_z_init`
- Change memory-mapped pointer target: `x_init_ptr` $\rightarrow$ `y_init_ptr`, `z_init_ptr`
- Update terminal print strings

Code Insertions

Handler for y initial condition

```
else if (strcmp(input, "y") == 0)
{
    float new_y_init = 0;
    printf("Enter new y_init: ");
    scanf("%f", &new_y_init);

    *y_init_ptr = (signed int)(new_y_init * SCALE);

    printf("New y_init is: %f\n", new_y_init);
}
```

Handler for z initial condition

```
else if (strcmp(input, "z") == 0)
{
    float new_z_init = 0;
    printf("Enter new z_init: ");
    scanf("%f", &new_z_init);

    *z_init_ptr = (signed int)(new_z_init * SCALE);

    printf("New z_init is: %f\n", new_z_init);
}
```

Explanation

The hardware solver operates using fixed-point arithmetic. Therefore, floating-point values entered in the terminal must be converted before being written to the FPGA registers:

$$\text{Fixed-point value} = \text{Floating-point value} \times \text{SCALE}$$

The HPS converts user input into an integer representation and writes the value into the memory-mapped register corresponding to the selected Lorenz state variable (x , y , or z).